A D PATEL INSTITUTE OF TECHNOLOGY
DEPARTMENT OF INFORMATION TECHNOLOGY

PASSWORD STRENGTH CHECKER

IN

Python

A mini project report submitted
In partial fulfilment of the requirements for the Degree of Bachelor of Technology

In
Information Technology
Semester – V

By

PATEL MAULI (12502080603011)

Faculty Guide
Asst Prof. Nayan Mali

Academic Year 2026-27
Department of Information Technology
AD Patel Institute of Technology

A D PATEL INSTITUTE OF TECHNOLOGY
DEPARTMENT OF INFORMATION TECHNOLOGY

CERTIFICATE

This is to certify that **Ms. Mauli Patel**, Enrollment Number **12502080603011**, a student of B.Tech in Information Technology at **A. D. Patel Institute of Technology**, has successfully completed the mini project entitled **"Password Strength Checker"** as part of the academic requirements for the degree program.
The project has been developed using Python programming language with CustomTkinter for modern graphical user interface development and re (Regular Expressions) for password criteria evaluations. It demonstrates real-time password security analysis, character pattern checks, strength score calculation, and interactive GUI handling.
The work presented in this report is submitted for academic evaluation. To the best of our knowledge, the project work has been carried out by the student under the guidance and supervision of the faculty member.

Guide	**Head Of Department**
Nayan Mali	**Dr.Priyang Bhatt**
(Assistant Prof.)	**(Associate Prof. & Head)**

Acknowledgement

I would like to express my sincere gratitude to **A. D. Patel Institute of Technology (ADIT)** and the Department of Information Technology for providing me with the opportunity to undertake the project titled **"Password Strength Checker"**.
I express my deep sense of gratitude to my faculty guide **Asst. Prof. Nayan Mali** and **Dr. Priyang Bhatt** (HOD, IT Department) for their valuable guidance, continuous encouragement, and technical suggestions throughout the project development.
I also extend my thanks to the technical staff and my peers for providing a supportive environment and necessary academic resources. Finally, I would like to thank my family for their constant motivation and backing during this work.

MAULI PATEL
(12502080603011)

ABSTRACT

In modern digital security, weak passwords remain one of the primary vulnerabilities exploited by attackers. The **Password Strength Checker** application is a desktop utility built using **Python** and the modern UI library **CustomTkinter**. The tool evaluates password robustness in real-time as users type.
By combining regular expressions (Regex) with standard security heuristics, the system checks for character length, uppercase letters, lowercase letters, numerical digits, and special characters. Dynamic feedback is provided through an interactive checklist, a color-coded strength bar, and visual indicators (Weak, Medium, Strong, Very Strong). A toggle feature allows users to reveal or mask their input. The project highlights practical GUI event binding, regular expression parsing, modular code architecture, and desktop security user experience (UX) design.

TABLE OF CONTENTS

Section	Title
1	Introduction & Project Overview
2	System Requirements & Technologies Used
3	System Design & Security Algorithms
4	Implementation & Code Explanation
5	Testing, Results & Conclusion

1. INTRODUCTION

1.1 Background & Problem Statement

With the rapid expansion of digital authentication services, user credentials remain the primary gateway to personal data and enterprise systems. Statistically, credential stuffing, brute-force dictionary attacks, and automated rainbow table lookups exploit passwords with low entropy or missing character diversity. End-users frequently select weak, predictable passwords due to a lack of immediate visual feedback during password composition.

1.2 Project Objectives

The Password Strength Checker application is a lightweight, responsive desktop tool constructed with Python. Utilizing standard event binding (<KeyRelease>), the app instantly parses input as the user types, evaluating security density and rendering color-coded progress bars and real-time validation checks.

- Provide instantaneous visual feedback during password composition to eliminate guesswork.
- Implement rule-checking logic for length, uppercase, lowercase, numbers, and special symbols.
- Display dynamic progress bar colors (Red, Amber, Green, Teal) reflective of strength tier.
- Provide visibility toggle buttons for password masking and unmasking (`👁` / `🙈`).

2. SYSTEM REQUIREMENTS & TECHNOLOGIES USED

2.1 Core Technologies

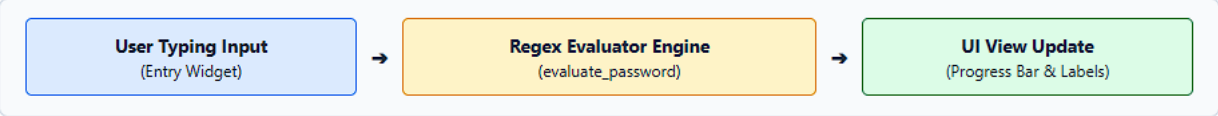
- **Python (v3.8+):** Serves as the core execution environment and application backbone. It handles object-oriented class instantiation, dynamic memory allocation, and synchronous event routing. Python's state-tracking mechanisms manage runtime data mutation, while its arithmetic engine processes real-time score computations and mathematical evaluations.
- **CustomTkinter (v5.0+):** Functions as the primary Graphical User Interface (GUI) framework. This modern wrapper extends the native Tkinter API to supply highly customizable, hardware-accelerated visual widgets. Key features include automated system-level dark/light mode detection, anti-aliased rounded frame rendering, and native high-DPI scaling across varying display densities.
- **Regular Expressions (re Standard Library):** Serves as the core text-parsing and validation engine. It executes deterministic pattern-matching operations against incoming string buffers. The engine isolates, filters, and validates targeted character classes, syntax structures, and tokens with an optimal time complexity of O(N), where N represents the length of the input stream.

Component	Specification / Requirement
Operating System	Windows 10/11, macOS 11+, or Linux (Ubuntu/Debian)
Python Version	Python 3.8 or higher
Dependencies	customtkinter (v5.0+), re (Standard Library)
Development IDE	Visual Studio Code / PyCharm / IDLE

3. SYSTEM DESIGN & SECURITY ALGORITHMS

3.1 System Architecture

The program follows an Event-Driven Object-Oriented Architecture. The main view inherits directly from `ctk.CTk`, encapsulating UI widgets and callback methods in a unified container class.



3.2 Password Strength Evaluation Logic

1. Retrieve raw string from password_entry using .get().
2. If string is empty: set score to 0, progress bar to 0, status to "Empty", and reset all rule labels to ❌ (red).
3. Execute dictionary containing 5 boolean evaluation expressions using regex.
4. Iterate through evaluation results: update matching label text and color (Green / ❌ Red), incrementing score count.
5. Apply score threshold map:
   - **Score ≤ 2:** Weak (Red, Progress: 0.25)
   - **Score 3 or 4:** Medium (Orange, Progress: 0.50)
   - **Score = 5 & Length ≤ 11:** Strong (Green, Progress: 0.75)
   - **Score = 5 & Length > 11:** Very Strong (Teal, Progress: 1.00)

3.3 Rule Set & Regex Mapping

Rule Key	Target Regex Pattern	Description Benchmark
length	len(pwd) >= 8	String length equal or exceeding 8 chars
upper	r"[A-Z]"	At least one uppercase alphabetical character
lower	r"[a-z]"	At least one lowercase alphabetical character
digit	r"\d"	At least one numerical digit (0-9)
special	r"[!@#$%^&*(),.?\":{}|<>]"	At least one symbol or special character

3.1 IMPLEMENTATION & CODE EXPLANATION

3.2 Hardware Requirements

- A desktop or laptop computer.
- A keyboard and pointing device.
- Basic memory and storage sufficient to run Python and save a small text file.
- A display capable of showing a standard desktop window.

3.3 Software Requirements

- Python 3.x.
- Tkinter support in the Python installation.
- An editor or IDE such as IDLE or Visual Studio Code.
- Windows, Linux, or macOS with a supported graphical desktop environment.

3.4 Modules and Responsibilities

Module / Component	Responsibility
tkinter	Creates the window and GUI controls such as labels, entry fields, list boxes, frames, and buttons.
messagebox	Displays warnings for empty input or missing task selection.
json	Reads and writes the task list using JSON format.
os	Checks whether the JSON storage file exists.
tasks list	Keeps task dictionaries in memory while the program runs.
tasks.json	Stores task records between application sessions.

3.5 Installation and Execution

1. Install a compatible Python 3 version.
2. Save the source code as todo_app.py.
3. Confirm Tkinter is available in the Python installation.
4. Run the file using an IDE or the command python todo_app.py.
5. Use the window to add, complete, and delete tasks.

4. IMPLEMENTATION & CODE EXPLANATION

4.1 UI Framework and Component Layout

The application utilizes an object-oriented design pattern where the core class inherits from the CustomTkinter root window class to manage the lifecycle and graphical canvas configurations.

```
class PasswordCheckerApp(ctk.CTk):
  def __init__(self):
    super().__init__()

    self.title("Password Strength Checker")
    self.geometry("450x520")
    self.resizable(False, False)

    self.show_password = False
```

- **super().__init__()**: Spawns the underlying graphical interface window context, establishing operating system handles and runtime systems.
- **Window Constraints**: The application window is set to a fixed boundary profile of **450x520 pixels**. Disabling resizing via self.resizable(False, False) guarantees that child UI widgets maintain strict layout positioning and structural integrity across execution environments.
- **State Properties**: The boolean property self.show_password tracks input security status, serving as a state-machine toggle flag for character masking.

Visual Containers and Controls Layout

```
    self.main_frame = ctk.CTkFrame(self, corner_radius=12)
    self.main_frame.pack(fill="both", expand=True, padx=20, pady=20)

    self.title_label = ctk.CTkLabel(
      self.main_frame,
      text="Password Strength Checker",
      font=ctk.CTkFont(size=22, weight="bold")
    )
    self.title_label.pack(pady=(25, 5))

    self.subtitle_label = ctk.CTkLabel(
      self.main_frame,
      text="Enter a password below to test its strength",
      font=ctk.CTkFont(size=12),
      text_color="gray"
    )
    self.subtitle_label.pack(pady=(0, 20))
```

- **self.main_frame**: Serves as the central visual enclosure box. It uses a modern 12-pixel corner roundness configuration and fills the root canvas space while maintaining strict perimeter padding.
- **Header Elements**: The title_label and subtitle_label establish a typographic visual scale by contrasting a large bold title font with a small, muted grey subtext string.

```
    self.input_frame = ctk.CTkFrame(self.main_frame, fg_color="transparent")
    self.input_frame.pack(fill="x", padx=30, pady=10)

    self.password_entry = ctk.CTkEntry(
      self.input_frame,
```

```
    placeholder_text="Enter password...",
    show="*",
    height=40,
    font=ctk.CTkFont(size=14)
)
self.password_entry.pack(side="left", fill="x", expand=True, padx=(0, 10))
self.password_entry.bind("<KeyRelease>", self.evaluate_password)

self.toggle_btn = ctk.CTkButton(
    self.input_frame,
    text="👁",
    width=40,
    height=40,
    font=ctk.CTkFont(size=16),
    command=self.toggle_password_visibility
)
self.toggle_btn.pack(side="right")
```

- **self.input_frame**: A transparent layout capsule that isolates horizontal alignment rules for the data-entry components.
- **self.password_entry**: The input interface element initialized with a hidden mode flag (show="*") to protect credentials. It handles real-time verification by anchoring an execution hook directly to the **<KeyRelease> asynchronous window event handler**. This design triggers password metric evaluations instantly after every character modification.
- **self.toggle_btn**: A fixed-dimension canvas button bound to the visibility utility method, displaying responsive icons based on current masking requirements.

```
self.progress_bar = ctk.CTkProgressBar(self.main_frame, height=8, corner_radius=4)
self.progress_bar.pack(fill="x", padx=30, pady=(15, 5))
self.progress_bar.set(0)

self.status_label = ctk.CTkLabel(
    self.main_frame,
    text="Strength: Empty",
    font=ctk.CTkFont(size=14, weight="bold"),
    text_color="gray"
)
self.status_label.pack(pady=5)
```

- **Indicators**: The self.progress_bar displays evaluation scores on a normalized decimal scale from 0.0 to 1.0. The status_label outputs literal classifications (such as "Weak" or "Strong") below the bar for clear user feedback.

Checklist Component Generation

```
self.checklist_frame = ctk.CTkFrame(self.main_frame, fg_color="transparent")
self.checklist_frame.pack(fill="both", expand=True, padx=40, pady=15)

self.rules = {
    "length": "At least 8 characters long",
    "upper": "Contains an uppercase letter (A-Z)",
    "lower": "Contains a lowercase letter (a-z)",
    "digit": "Contains at least one number (0-9)",
    "special": "Contains a special character (@, #, $, etc.)"
}
```

```
self.labels = {}
for key, text in self.rules.items():
    lbl = ctk.CTkLabel(
        self.checklist_frame,
        text=f"❌ {text}",
        anchor="w",
        font=ctk.CTkFont(size=13),
        text_color="#e74c3c"
    )
    lbl.pack(fill="x", pady=4)
    self.labels[key] = lbl
```

- **Dynamic Generation Loop**: The script iterates through a static criteria schema (self.rules) to assemble interface elements dynamically. This process initializes all security requirements in a uniform format, mapping them with negative status badges (❌), setting an explicit red typography color (#e74c3c), and saving the structural references into an internal tracking container (self.labels) for later state changes.

4.2 Dynamic Control Controllers and Evaluation Logic

The application coordinates interface changes and input filtering using background state controllers and regular expression search processes.

Component Masking Switch

```
def toggle_password_visibility(self):
    if self.show_password:
        self.password_entry.configure(show="*")
        self.toggle_btn.configure(text="👁")
        self.show_password = False
    else:
        self.password_entry.configure(show="")
        self.toggle_btn.configure(text="🙈")
        self.show_password = True
```

- This method checks the internal self.show_password visibility flag to dynamically update the input entry box configurations. It toggles masking between safe display layouts (*) and plain-text output modes, while simultaneously swapping button icons to match the active state.

Interface State Management and Pattern Evaluation

```
def evaluate_password(self, event=None):
    pwd = self.password_entry.get()

    if not pwd:
        self.progress_bar.set(0)
        self.progress_bar.configure(progress_color="gray")
        self.status_label.configure(text="Strength: Empty", text_color="gray")
        for lbl in self.labels.values():
            lbl.configure(text=f"❌ {lbl.cget('text')[3:]}", text_color="#e74c3c")
        return
```

- **Empty Field Interception**: The system retrieves the text string using the entry field interface. If the value is empty, the method resets the tracker bar and status elements to grey. It safely isolates user-facing string descriptions using a **slice technique ([3:])**,

stripping off active icon headers to clean and rebuild standard layout strings without needing hardcoded resets.

```
checks = {
   "length": len(pwd) >= 8,
   "upper": bool(re.search(r"[A-Z]", pwd)),
   "lower": bool(re.search(r"[a-z]", pwd)),
   "digit": bool(re.search(r"\d", pwd)),
   "special": bool(re.search(r"[!@#$%^&*(),.?\":{}|<>]", pwd))
}
```

- **Pattern Evaluation Logic**: The input is tested against five security constraints. By passing regular expression lookup results into **bool()**, raw character index matches or empty search outcomes map cleanly into reliable True or False validation flags:
    - length: Determines if total string array counts meet or exceed the 8-character baseline rule.
    - upper: Evaluates character distributions against uppercase boundaries ([A-Z]).
    - lower: Evaluates character distributions against lowercase boundaries ([a-z]).
    - digit: Employs a regex shorthand digit class (\d) to trace numerical character hits.
    - special: Runs pattern-matching queries using a designated set of standard ascii symbol marks.

```
score = 0
for key, passed in checks.items():
   lbl = self.labels[key]
   text = self.rules[key]
   if passed:
      lbl.configure(text=f"✓  {text}", text_color="#2ecc71")
      score += 1
   else:
      lbl.configure(text=f"❌  {text}", text_color="#e74c3c")
```

- **Visual Update Loop**: An iteration loop scans the boolean checks dictionary. Favorable validation flags update requirements with positive symbols (✓) and green coloring (#2ecc71) while incrementing the core score. Failed validations default to negative indicators (❌) and red styling (#e74c3c).

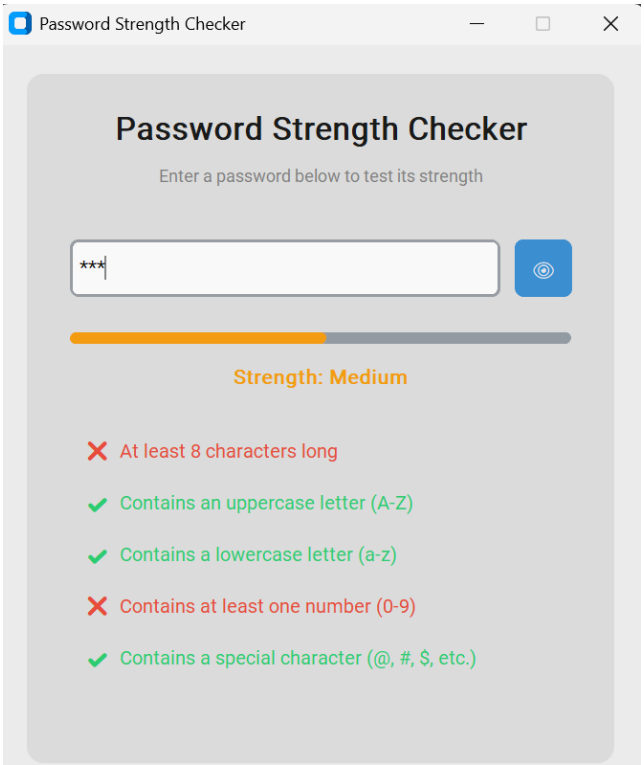
```
if score <= 2:
   self.status_label.configure(text="Strength: Weak", text_color="#e74c3c")
   self.progress_bar.configure(progress_color="#e74c3c")
   self.progress_bar.set(0.25)
elif score <= 4:
   self.status_label.configure(text="Strength: Medium", text_color="#f39c12")
   self.progress_bar.configure(progress_color="#f39c12")
   self.progress_bar.set(0.5)
elif score == 5 and len(pwd) <= 11:
   self.status_label.configure(text="Strength: Strong", text_color="#2ecc71")
   self.progress_bar.configure(progress_color="#2ecc71")
   self.progress_bar.set(0.75)
elif score == 5 and len(pwd) > 11:
   self.status_label.configure(text="Strength: Very Strong", text_color="#1abc9c")
   self.progress_bar.configure(progress_color="#1abc9c")
   self.progress_bar.set(1.0)
```

Score-to-UI Mapping: The application evaluates the total accumulated score against a multi-tiered security profile matrix to apply final UI properties:

- **Scores ≤ 2**: Classified as **Weak**. Applies a red visual theme and scales the tracker fill metric to 0.25.
- **Scores of 3 or 4**: Classified as **Medium**. Applies an orange visual theme (#f39c12) and scales the tracker fill metric to 0.50.
- **Scores of 5 (Length ≤ 11)**: Classified as **Strong**. Applies a green visual theme (#2ecc71) and scales the tracker fill metric to 0.75.
- **Scores of 5 (Length ≥ 12)**: Classified as **Very Strong**. Fulfilling all individual structural criteria and scaling beyond a 11-character boundary unlocks the maximum security tier. This applies a teal visual theme (#1abc9c) and sets the progress bar fill to its maximum value (1.0).

Application Startup Block

```
if __name__ == "__main__":
    app = PasswordCheckerApp()
    app.mainloop()
```



5. TESTING, RESULTS & CONCLUSION

ID	Test Input	Tested Condition	Expected Result	Status
TC01	"" (Empty)	Zero-length handling	Score: 0, Gray, All ❌	PASS
TC02	"abc"	Short lowercase string	Weak, Red Bar (25%)	PASS
TC03	"Abc12345"	8+ chars, upper, lower, digit	Medium, Orange Bar (50%)	PASS
TC04	"P@ssword1"	All 5 rules satisfied (len 9)	Strong, Green Bar (75%)	PASS
TC05	"P@ssword12345"	All 5 rules satisfied (len 13)	Very Strong, Teal Bar (100%)	PASS

5.1 Conclusion & Future Scope

The Password Strength Checker project successfully demonstrates how desktop GUI applications in Python can be designed with contemporary aesthetics and instant validation responsiveness. Future enhancements include integrating the zxcvbn entropy calculation library and adding an automated password generator tool.

5.2 References

- Python Software Foundation. *re — Regular expression operations*. https://docs.python.org/3/library/re.html
- CustomTkinter Documentation. *Modern UI library for Python*. https://customtkinter.tomschimansky.com/